

VIETNAM NATIONAL UNIVERSITY, HO CHI MINH CITY
UNIVERSITY OF SCIENCE
FACULTY OF INFORMATION TECHNOLOGY

Dương Trung Hiếu - Dương Ngọc Quang Khiêm
22125027 - 22125037

uMatter: An Application for Supporting Mental Health

GRADUATION THESIS
BACHELOR OF SCIENCE IN COMPUTER SCIENCE

Advisor: PGS.TS Nguyễn Văn Vũ

Ho Chi Minh City, 2026

Statement of Originality

I hereby declare that this is my own research, carried out under the supervision of PGS.TS Nguyễn Văn Vũ. The data and results presented in this thesis are truthful and have not been duplicated from any other work.

Acknowledgments

We would like to sincerely thank our advisor for their guidance throughout this thesis, the Faculty of Information Technology at the University of Science (VNU-HCM) for the resources and feedback that shaped this work, and our families and friends for their continued support during the development of uMatter.

Table of Contents

Acknowledgments	ii
Table of Contents	iii
List of Tables	vi
Abstract	vi
1 Introduction	1
1.1 Background and Motivation	1
1.2 Problem Statement	4
1.3 Objectives	6
1.4 Scope	7
1.5 Contributions	8
2 Related Work	9
2.1 Self-Care and Tracking Applications	9
2.2 Professional Therapy Platforms	10
2.3 Social and Family Connection Platforms	10
2.4 General-Purpose AI for Mental Health Advice	11
2.5 Positioning of uMatter	12
2.6 Architectural Foundations	12
2.7 Security Foundations	13
2.8 AI Companion And Video Consultation	13

3	System Analysis and Design	15
3.1	Requirements Analysis	15
3.2	Overall Architecture	17
3.3	Service Decomposition	18
3.4	Data Architecture	18
3.5	Event-Driven Messaging	20
3.6	Authentication, Authorization, and Data Sharing	22
3.7	AI Companion	24
3.8	Professional Workflows	25
3.9	Peer and Family Connection	26
3.10	Self-Care Features	27
3.11	Deployment Topology	28
4	Implementation and Evaluation	29
4.1	Implementation Overview	29
4.2	Deployment	30
4.2.1	Mobile Application Distribution	31
4.3	Security Hardening	31
4.4	Evaluation	33
4.4.1	Functional Completeness	33
4.4.2	Security Posture	33
4.4.3	Comparison with Existing Platforms	34
4.4.4	Most-Valued Features	34
4.4.5	Key Findings	35
5	Conclusion and Future Work	38
5.1	Conclusion	38
5.2	Limitations	39
5.3	Future Work	39
	References	40

A	API Endpoint Summary	43
B	Deployment Configuration Notes	45
C	Requirement Traceability	46

List of Tables

2.1	Feature comparison between uMatter and comparable platforms	14
3.1	Key functional requirements and the objectives they realize	16
3.2	Key non-functional requirements	17
3.3	uMatter backend services and their responsibilities	19
A.1	Representative endpoints by service	44
C.1	Implementation status of the requirements in Section 3.1 .	47

Abstract

Adolescent mental health support remains fragmented: self-care tools including mood or sleep trackers such as Daylio, Wysa and Bearable, rarely connect to licensed professional care, while therapy platforms such as BetterHelp and Talkspace typically specialize in only finding you a trusted therapist without giving the therapist enough context as to how you are doing in order to efficiently conduct meaningful sessions. And while there are platforms for people to have deep conversations with close friends and family, it is hard for parents to actually know how their children are doing and feeling. And as a trend of young people seeking mental health advice from AI, a centralized platform where the user can choose to share their daily reflective tracking data with AI would be more beneficial in case they truly want a deeply contextual advice instead of a vague one. This gap motivates a platform that unifies these fragmented features so that the user don't have to spend unnecessary time explaining themselves every time they want to use a feature from a different platform, providing the best mental health support as we can.

This thesis presents uMatter, a mental health support platform for teenagers that combines self-care features, an AI companion, peer social features as well as professional therapy features.

uMatter is implemented as a microservices system: seven Spring Boot services (authentication, tracking, AI, dashboard, therapist, social, and notification) sit behind an Nginx API gateway, communicate asynchronously through RabbitMQ, and follow a database-per-service pattern backed by

PostgreSQL, Redis, and MinIO. Access control uses stateless JWT authentication with a JWKS-published RSA key and a permission-based data-sharing model so users can selectively grant professionals access to their tracked data. The AI companion is grounded in each user’s own tracking history and served through Google’s Gemini 2.5 Flash. The system was hardened through structured security audits covering injection risks, insecure direct object references, and mass assignment, and a database-level atomic slot claim, backed by a unique constraint, was introduced to close a race condition in appointment booking. The full stack was deployed and validated on cloud infrastructure, running as 20 containers.

The resulting platform demonstrates that a unified self-care and professional therapy experience can be successfully delivered through a single, security-conscious microservices architecture. However, as an academic prototype, uMatter scopes out key real-world operational challenges. It excludes legal and compliance frameworks, dispute resolution mechanisms for handling user or therapist conflicts, and the monetization model essential for incentivizing licensed professionals. Despite these out-of-scope administrative limitations, this thesis establishes a robust technical foundation. Future work can build upon this architecture by integrating monetization systems, comprehensive legal safeguards, clinical collaboration requirements, and layered risk detection, ultimately advancing uMatter toward a fully viable ecosystem for adolescent mental health support.

Chapter 1

Introduction

1.1 Background and Motivation

Adolescent mental health in Vietnam has become a problem that needs more attention than ever. In recent years a ninth-grader in Ho Chi Minh City took his own life after a poor exam result [1], and a final-year Information Technology student at the University of Science, VNU-HCM died the same way [2]. These are not isolated: a cross-sectional study of 1,316 high-school students across six districts of Ho Chi Minh City found that 46.5% reported self-injury behaviour [3], and the global picture agrees the World Health Organization estimates that one in seven adolescents aged 10-19 experiences a mental disorder, accounting for 15% of the disease burden in this age group, with suicide the third leading cause of death among those aged 15-29 [4]. What makes such cases so difficult to accept is that the distress is rarely invisible in hindsight. It is usually present beforehand, in ordinary conversations and small changes of behaviour, and simply never seen by anyone positioned to act. The signal exists; nothing is holding it.

Yet the tools available to a teenager today address only fragments of this problem. On one side of the market, self-care applications such as Daylio and Bearable let users log their mood, sleep, and daily activities, helping them notice patterns in their own well-being [5], [6]. These tools are lightweight and approachable, but they stop at self-observation: they

do not connect a user who needs help to a licensed professional. On the other side, therapy-oriented platforms such as BetterHelp and Talkspace connect users with licensed therapists for text, audio, or video counseling [7], [8], while AI-first platforms such as Wysa offer a conversational companion alongside optional access to coaches [9]. These platforms are targeted primarily around adults, and around one side of the care journey, either self-tracking or professional therapy. The consequence is that a therapist meets a patient with no context beyond what that patient can reconstruct from memory in the room, and a parent has no way of knowing how their child is actually doing.

What we learned from users

To find where these tools fail in practice, we ran a baseline study in which a psychology student at HCMUE, a Business student at RMIT, evaluated ten existing platforms: AI chatbots, trackers, peer communities, telehealth services, and clinical software against two personas: an undergraduate diagnosed with mild-to-moderate depression, and a tenth-grader under academic pressure whose family does not believe anything is wrong. Four findings shaped this thesis. First, the week between sessions is a black box: by the appointment, what the patient meant to discuss is forgotten or too exhausting to reconstruct. Second, input fatigue was the single most cited reason for abandoning a health app. Third, tracking alone produced no progress; trackers were judged useful for noticing patterns and worthless for changing them, while an AI companion was seen as genuinely supportive but never a substitute for a human professional. Fourth, privacy is a precondition, not a feature: the strongest reaction concerned data leaking at the moment the user is most vulnerable, and willingness to share was never absolute a journal could go to a therapist but not to friends, and an automatic crisis alert to family was rejected outright as pressure to perform a better mood than the one being felt.

What we learned from professionals

We then consulted three mental-health professionals: a counsellor from the Psychological Counselling Office at the University of Science, VNU-HCM; the CEO of the psychology organization LUMOS; and a physician at Hoàn Mỹ Sài Gòn Hospital. Their guidance shaped the product directly: replacing periodic surveys with a continuous emotion timeline captured every few hours, because daily reflection yields richer and more meaningful data than any questionnaire; routing a user who logs anger toward a breathing exercise and a user who logs sadness toward their stored positive memories; and treating physical activity, sleep, and nutrition as inseparable from mental well-being.

Two of the three were substantially opposed to AI in mental health: one holding that an AI should answer only the easiest questions and should never be given access to a user's personal tracking data at all, while the third supported it, carefully applied, as a genuine source of companionship. One was openly distrustful of peer connection, on the grounds that a well-meaning friend may offer advice that runs directly against clinical guidance, and recommended that such connections be limited to parents and preceded by explicit warnings. As a result, every audience including the AI companion, a therapist, a trusted friend or parent, must be granted access explicitly, in scope, and revocably, with nothing shared by default.

Why we built this

Finally, and most plainly: the two of us wanted to help. We are not clinicians, and this thesis does not pretend to treat anyone. But we can build software, and much of what is missing is a software problem somewhere for a young person's daily reality to be captured comfortably enough that it actually gets captured, and a controlled path from that record to the people who can help, whether that is an AI at 3 AM, a friend, a parent, or a licensed therapist.

1.2 Problem Statement

Problem 1: Eliciting the data

How can a platform obtain an honest, continuous record of a teenager's mood, emotions, and daily wellbeing in a way that feels natural and comfortable rather than like clinical paperwork?

The ideal solution would be *implicit*: inferring emotional state from signals the user already produces, without asking them to report anything. The motivating case is: before a recent suicide by a secondary-school student in Ho Chi Minh City [1], the warning was present in ordinary conversations with friends; had it been noticed, it might have been acted upon. But capturing that class of signal means continuously listening to real-life conversations, reading messaging apps, or monitoring facial expressions around the clock. Such surveillance is technically fragile, ethically indefensible for minors, and in direct conflict with data-protection law. That decision converts Problem 1 into a design problem: self-report is only useful if it actually happens, day after day, and self-report is exactly what users abandon first. The difficulty is threefold:

- **Friction.** Every second of effort between "I feel something" and "it is recorded" costs data. Capture must be a single tap, expanded only if the user wants to elaborate.
- **Recall and initiation.** A teenager in a low mood is the least likely person to remember to open a wellbeing app. The system must come to the user rather than wait for them, without becoming a nag that gets muted or uninstalled.

This thesis addresses Problem 1 through a set of **daily reflection and tracking** features designed as one habit rather than ten tools: a one-tap **mood check-in** (*Cảm xúc lúc này*) across five levels with an optional line

of context; an **emotional journal** (*Góc tâm tư*) for free-form reflection with photos, a mood calendar, and streaks; and **wellbeing logs** for sleep, nutrition and water, and breathing with step count collected automatically, the one passive signal that requires no surveillance of the user’s private life. Elicitation itself is handled by **Focus Mode**, which nudges roughly every 90 minutes during waking hours and respects a Quiet Hours window, so the record accumulates without the user having to remember to build it.

Problem 2: Acting on the data

Given that record, how can the platform tell whether a user is in distress, act appropriately when they are, and still be worth opening when they are not? Resolving this requires nuanced interpretation to distinguish acute distress from an ordinary bad day or a long-term downward trend. Furthermore, because most days are simply neutral or mildly negative, a platform built only for crises will remain closed on an average day. Finally, navigating these states safely demands strict data sensitivity, ensuring that any access to a user’s mood, sleep, and journal data is always explicit and revocable, rather than granted by default.

In crisis, a dedicated emergency-support area always one tap away. For the ordinary day, the same record powers an AI companion whose replies are grounded in how the user has actually been sleeping and feeling, guided breathing and meditation, a Treasure Box of pre-saved positive memories to reopen when recall of the good is hardest, and lightweight achievements that make the habit itself rewarding.

The two problems close a loop: better daily reflection makes every downstream response more precise, and responses that are genuinely useful on ordinary days are what keep the reflection going.

1.3 Objectives

Capturing the record (Problem 1)

- O1 Make capture experience as comfortable as possible.
- O2 Bring the system to the user without becoming a nag.
- O4 Cover physical wellbeing as the consulted professionals framed it

Acting on the record (Problem 2)

- O5 Ground the AI companion in the user's own history
- O6 Respond appropriately to the ordinary day, through guided breathing and meditation, a Treasure Box of pre-saved positive memories surfaced when recall of the good is hardest, emotion-appropriate routing (anger toward breathing, sadness toward stored memories), and lightweight achievements and streaks that make the habit itself rewarding.
- O7 Keep the crisis path one tap away
- O8 Close the black box between sessions

Consent, architecture, and validation

- O9 Share nothing by default.
- O10 Build the system as a maintainable microservices architecture
- O11 Demonstrate that the result runs, by deploying and validating the full stack on cloud infrastructure.

1.4 Scope

uMatter is built as an academic prototype: a working system, deployed and exercised end to end, whose purpose is to demonstrate that the unified experience can be delivered through a single coherent architecture. Its intended users are Vietnamese adolescents, with the audiences they may choose to admit, an AI companion, a licensed therapist, a parent, or a friend. Within that, the thesis covers the daily reflection and tracking features, the responses built on top of them, the permission model governing who may see what, the seven-service backend and its supporting infrastructure, and the deployment of the whole stack to cloud infrastructure.

Several concerns that a production deployment would have to address are deliberately excluded, and are treated in future work rather than as gaps in the implementation:

- **Legal and regulatory compliance.** Data-protection obligations, consent law for minors, and the licensure requirements governing teletherapy are outside the scope of this thesis.
- **Dispute resolution.** The platform provides no mechanism for adjudicating conflicts between a user and a therapist, or between users.
- **Monetization.** No payment, billing, or compensation model is implemented. This is not a missing feature: without it there is no incentive structure to attract licensed professionals to the platform.
- **Clinical validation.** uMatter makes no diagnostic claim and was not evaluated in a clinical trial. No assertion is made that the platform improves mental-health outcomes; the evaluation concerns the system, not its therapeutic efficacy.
- **Automated risk detection.** The platform does not attempt to infer crisis from the tracked record and does not escalate on the user's

behalf. The emergency-support path is user-initiated by design, a decision reinforced by our baseline study, in which an automatic crisis alert to family was rejected outright as pressure to perform a better mood than the one being felt.

1.5 Contributions

This thesis makes the following contributions:

- **A unified platform.** uMatter integrates daily reflective tracking, an AI companion, peer and family connection, and professional therapy into a single system, so that a user need not re-explain themselves each time they cross from one fragment of the existing market into another.
- **A consent model.** The permission-based data-sharing design, in which every audience is granted access explicitly, in scope, and revocably, with nothing shared by default.
- **Requirements grounded in primary study.** The design is driven by a baseline evaluation of ten platforms against two personas, conducted with a psychology student at HCMUE, and by consultation with three mental-health professionals. The findings are reported and traced to the features they produced.
- **A pioneering localized prototype.** As one of the first native applications tailored specifically for adolescent mental health in Vietnam, uMatter raises critical awareness about the well-being of Vietnamese teenagers and young adults. It provides a meaningful, usable prototype demonstrating that mental health can be supported in many connected ways, with the help of yourself and the people around who truly care about you, embodying the platform’s core message: *you matter*.

Chapter 2

Related Work

The Abstract and Chapter 1 describe adolescent mental-health support today as fragmented across four kinds of tools. This chapter surveys each of these four fragments in turn, positions uMatter relative to them, and reviews the architectural and security literature that informs the design presented in Chapter 3.

2.1 Self-Care and Tracking Applications

Daylio and Bearable are representative of a category of lightweight, single-purpose self-care applications [5], [6]. Daylio focuses on quick, low-friction mood and activity logging, encouraging habit formation through streaks and reminders. Bearable extends this idea to a broader set of symptoms and factors (sleep, medication, exercise, weather, and more), letting users explore correlations in their own data.

Both applications are valuable for self-observation, but neither connects a user to a professional: there is no booking, matching, or clinical workflow, and the data collected has no path out of the app other than manual export. For a teenager who identifies a concerning pattern in their own tracked data, these tools offer no next step, and our own baseline study found the same limitation from the user’s side: trackers were judged useful for noticing patterns and worthless for changing them.

2.2 Professional Therapy Platforms

BetterHelp and Talkspace sit at the opposite end of the spectrum [7], [8]. Both connect users with licensed therapists through a subscription model, primarily via asynchronous messaging with optional live sessions. Their onboarding is built around a short intake questionnaire used to match a new user to a therapist, and their core workflows (scheduling, secure messaging, video sessions) are mature and well-tested at scale.

These platforms are built for a general adult audience, and they do not provide day-to-day self-care tracking as a first-class feature - a user's mood or sleep history is not something the platform helps them build up over time outside of a therapy session. A therapist on these platforms meets a patient with no context beyond what that patient can reconstruct from memory in the room, which is precisely the black-box week our baseline study identified as a recurring failure. These platforms also do not target the specific constraints of a teenage user base, such as guardian involvement or age-appropriate content and matching.

2.3 Social and Family Connection Platforms

A third fragment is not a mental-health product at all, but the general-purpose messaging and social platforms a teenager already uses to talk to friends and family. In Vietnam, this is overwhelmingly Zalo, the country's leading messaging platform [10], alongside global platforms such as Messenger. These platforms are conversational infrastructure, not wellbeing infrastructure: they carry whatever a teenager chooses to say, but they do not structure that conversation around mood or wellbeing, and they give a parent no visibility into how their child is actually doing beyond what that child volunteers to share.

Family-oriented applications such as Life360 address a related but different problem, surfacing a family member's location and presence rather

than their emotional state [11], so a parent can confirm that their child arrived home safely without learning anything about how the child felt that day. This is precisely the gap our professional consultations surfaced: a parent has no way of knowing how their child is actually doing. It is also why our own consulted professionals disagreed on how far peer and family connection should go at all, one warning that a well-meaning friend’s advice can run against clinical guidance, which is why uMatter treats a parent or friend as an audience that must be explicitly and revocably granted access, rather than a channel that is open by default.

2.4 General-Purpose AI for Mental Health Advice

A fourth fragment is not a product category but a documented behavior: adolescents and young adults increasingly turn to general-purpose AI chatbots, ChatGPT, Google Gemini, Meta AI, and similar tools, when they feel sad, angry, or nervous. A nationally representative U.S. survey found that roughly one in eight adolescents and young adults aged 12 to 21 had used a generative-AI chatbot for mental health advice, with the large majority of those users rating the advice helpful [12]. Wysa is the most mature purpose-built instance of this same pattern, wrapping a conversational AI companion around a mental-health-specific design, with an optional paid tier that adds access to human coaches [9].

What both the general-purpose chatbots and Wysa share, from uMatter’s perspective, is that neither is grounded in a continuous, structured record of the user’s own life: a general-purpose chatbot starts from nothing every conversation, and Wysa’s companion, while mental-health-aware, does not draw on a user’s own logged mood, sleep, or journal history at all. uMatter’s own companion can draw on that history, but only once the user has explicitly granted it access through the same permission model

used for a therapist (Section 3.6); absent that grant, it converses without grounding rather than reading tracking data by default. The same survey work that documents this trend also cautions that there are few standardized benchmarks for the quality of the mental-health advice these systems provide, which echoes what our own consulted professionals insisted on: an AI companion should never substitute for a human professional, and, for at least one of the three we consulted, should not be given access to a user’s personal tracking data at all.

2.5 Positioning of uMatter

Table 2.1 summarizes this comparison across all four fragments surveyed above: self-care tracking, professional therapy, social and family connection, and AI-based advice-seeking. uMatter’s distinguishing position is that it treats these as four aspects of one continuous user journey rather than four separate products a teenager must adopt and re-explain themselves, connected by a permission-based data-sharing model in which a user’s own tracked history travels only where they explicitly allow it: to a matched therapist, to a parent or friend, or to the AI companion itself, with nothing shared by default.

2.6 Architectural Foundations

The system architecture proposed in Chapter 3 builds on well-established distributed-systems practice rather than novel architectural research. The decomposition into independently deployable services, each owning its own data store, follows the microservices style described by Newman [13], chosen over a monolithic design because uMatter’s functional areas (authentication, tracking, AI, professional workflows, social features, notifications) have different scaling, security, and change-rate profiles. Inter-service communication uses a mix of synchronous REST calls, following the archi-

tectural constraints of the REST style [14], and asynchronous messaging through RabbitMQ for event-driven workflows [15]. Services are implemented with Spring Boot [16] and packaged with Docker [17] to keep the development and deployment environments consistent.

2.7 Security Foundations

Securing a distributed system requires standards-based authentication and proactive threat modeling. Modern architectures favor stateless, token-based authentication via JSON Web Tokens (JWT) [18], verified cryptographically through JSON Web Key Sets (JWKS) [19] to eliminate centralized session bottlenecks. System design is typically benchmarked against consensus frameworks like the OWASP Top 10 [20] to proactively mitigate critical vulnerabilities such as Broken Access Control and Injection flaws.

Beyond authentication, robust authorization relies on formal access control models. While Role-Based Access Control (RBAC) [21] efficiently manages coarse-grained permissions, Attribute-Based Access Control (ABAC) [22] is necessary for fine-grained, ownership-driven data sharing. To enforce these models safely, security literature emphasizes the principle of Defense in Depth [23]. Access rules are embedded declaratively at the method level.

2.8 AI Companion And Video Consultation

The in-app AI companion is powered by the Gemini API from Google, utilizing the Gemini 2.5 Flash model [24]. Unlike a general-purpose chatbot, the companion is grounded in the user’s own tracking history, so its responses can refer to a user’s recently logged mood, sleep, or journal entries rather than operating from a blank context. Video consultation between users and therapists is facilitated through Jitsi Meet [25] public meeting rooms on the internet, providing an open-source video-conferencing solution that avoids dependence on a proprietary video vendor.

Table 2.1: Feature comparison between uMatter and comparable platforms

Feature	uMatter	Better- Help / Talkspace	Wysa / General AI Chatbots	Daylio / Bearable
Mood/sleep/journal tracking	Yes	No	Partial	Yes
AI companion grounded in own data	Yes	No	No (general)	No
Licensed therapist matching & booking	Yes	Yes	Limited	No
Video consultation	Yes	Yes	No	No
Clinical notes for professionals	Yes	Yes	No	No
Peer social features	Yes	No	No	No
Parent/guardian visibility (with consent)	Yes	No	No	No
Teen-oriented design	Yes	No	Partial	Partial
Granular, permission-based data sharing	Yes	N/A	N/A	N/A

Chapter 3

System Analysis and Design

This chapter outlines uMatter’s requirements, microservices architecture, and shared technical foundations. It then details the core features (the AI companion, therapist workflows, social connections, and self-care) and concludes with the deployment topology evaluated in Chapter 4.

3.1 Requirements Analysis

uMatter has three direct actors: the primary user, who can be the teenager seeking support for mental health or a close friend or family member, and who tracks their wellbeing on the mobile application and owns every piece of data they produce; the therapist, working from the web dashboard; and the administrator. Importantly, a teen may admit others to their tracked record: therapists, parents and close friends (other user accounts), and the AI companion, all subject to the same grant model (Section 3.6).

Table 3.1 lists the key functional requirements distilled from these actors’ needs, and Table 3.2 lists the key non-functional requirements that constrain how they are implemented; both are traced back to the design objectives introduced in Chapter 1.

Table 3.1: Key functional requirements and the objectives they realize

ID	Requirement	Obj.
FR1	One-tap five-level mood check-in with optional context; journal with photos, mood calendar, and streaks.	O1, O6
FR2	Logs for sleep, nutrition and water, and breathing; step count is collected automatically and is the only passive signal.	O3, O4
FR3	Focus Mode prompts a check-in roughly every 90 minutes during waking hours, silent during user-configured Quiet Hours.	O2
FR4	The AI companion grounds its replies in the user’s own tracking data if and only if the user has granted it access.	O5, O9
FR5	Guided breathing and meditation, and a Treasure Box of saved positive memories, routed to by logged emotion; an emergency area reachable in one tap from anywhere.	O6, O7
FR6	Therapist discovery, appointment booking, video consultation, and clinical notes; tracked data visible to a therapist only under an explicit, scoped, revocable grant.	O8, O9
FR7	Friend and parent connections with real-time chat; a connection by itself exposes no tracked data.	O9
FR8	Domain events produce push, email, and in-app notifications to the affected parties.	O2, O8

Table 3.2: Key non-functional requirements

ID	Requirement	Obj.
NFR1	Privacy by default. No tracked data is visible to any audience without an active grant, enforced server-side so a compromised client cannot bypass it.	O9
NFR2	Stateless security. RSA-signed JWTs validated locally by every service against a published JWKS key.	O10
NFR3	Integrity under concurrency. Concurrent bookings of the same slot must not both succeed; retried requests are not applied twice.	O8
NFR4	Independent evolvability on modest infrastructure. Each service develops, deploys, and migrates independently, and the full platform runs on a single cloud virtual machine.	O10, O11

3.2 Overall Architecture

uMatter follows a microservices architecture, shown in Figure 3.1: independently deployable Spring Boot services sit behind an edge reverse proxy (Caddy) that terminates HTTPS at the system boundary and is the only entry point reachable by the two client applications: a React Native mobile app for teenagers/patients and a React web dashboard for therapists. Caddy serves the static web dashboard at the same origin and forwards STOMP/WebSocket chat straight to the Social service ; all REST traffic is passed to an internal Nginx API gateway, which routes each request to the owning backend service and applies central CORS.

This decomposition is not free, and for a two-person academic project the choice deserves justification. We accept the operational overhead of seven deployables, a message broker, and per-service databases in exchange for two things. First, independent lifecycles: the services live in separate

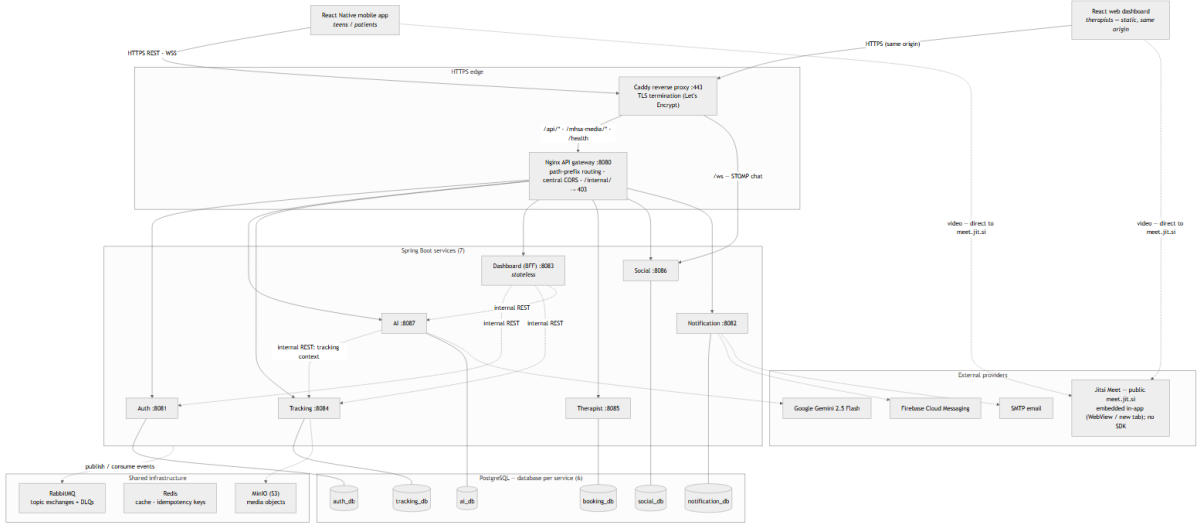


Figure 3.1: The overall microservices architecture of uMatter.

repositories and evolve without coordinated upgrades. Second, each service is scoped to a single functional area, can be developed, deployed, and scaled independently, and owns its own data.

This decomposition follows the pattern described by Newman [13]. Synchronous request/response interactions between services and clients use REST-style HTTP APIs [14]; workflows that are naturally asynchronous, such as sending a notification after an event happens elsewhere in the system, are handled through RabbitMQ instead [15].

3.3 Service Decomposition

The backend is organized into seven services, summarized in Table 3.3.

3.4 Data Architecture

uMatter follows a database-per-service pattern: each stateful backend service owns a private PostgreSQL database, and no service reaches directly into another service’s schema. Cross-service data needs are satisfied

Table 3.3: uMatter backend services and their responsibilities

Service	Responsibility
Auth	User registration/login, JWT issuance and refresh, JWKS key publication, role management
Tracking	Mood, sleep, and journal entry storage; the source of truth for a user’s self-reported wellbeing data
AI	Manages conversations with the AI companion, grounding responses in the user’s own tracking data
Dashboard	Aggregates tracking data into summaries and visual trends for the user
Therapist	Therapist discovery/matching, appointment booking, video consultation session management, clinical notes
Social	Peer relationships (friends) and real-time chat over STOMP/WebSocket
Notification	Consumes domain events and delivers push (FCM), email, and in-app inbox notifications

either through a synchronous API call to the owning service or through an asynchronous event, never through shared tables. This keeps each service free to evolve its own schema without coordinating a migration across the whole system, at the cost of needing an explicit strategy (API calls or events) for any data that spans services.

Two supporting stores complement PostgreSQL:

- **Redis** is used for caching and for idempotency keys, so that a retried request (for example, a client retry after a network timeout) is not applied twice.
- **MinIO**, an S3-compatible object store, holds binary media such as

profile photos and any files attached to journal entries or clinical notes, keeping large binary payloads out of the relational databases.

3.5 Event-Driven Messaging

RabbitMQ [15] connects services that should not depend on each other synchronously, and it plays two roles in uMatter: event-driven replication of data that one service owns but another needs, and fan-out of domain notifications to the parties an event affects.

To prevent synchronous bottlenecks when one service frequently requires data owned by another, the system uses an event-driven replication strategy via RabbitMQ topic exchanges. As illustrated in Figure 3.2, the architecture maintains four primary replication streams: the Therapist API replicates auth-owned therapist profiles; the Tracking Service maintains a local replica of data access grants (serving as the consent gate for patient data); the Auth service replicates patient-therapist assignments; and the Social service listens for active assignments to automatically provision direct chat channels between patients and their therapists.

To keep the replicas correct across these asynchronous boundaries, every consumer applies the same resilience discipline. Messages are acknowledged manually, and any that repeatedly fail are routed to a dead-letter exchange and queue (DLX/DLQ) for inspection and replay rather than blocking the consumer. Duplicate deliveries are absorbed by watermark idempotency on event time (an event no newer than the last one applied is skipped), and a nightly reconciliation against the owning service repairs any drift a prolonged outage may leave, ensuring eventual consistency.

Beyond replication, domain events also drive user-facing notifications. As shown in Figure 3.3, the Therapist service publishes `appointment.booked` when a patient books a session; the Notification service consumes it and fans the confirmation out to the affected user as an in-app inbox entry, an email, and a push notification.

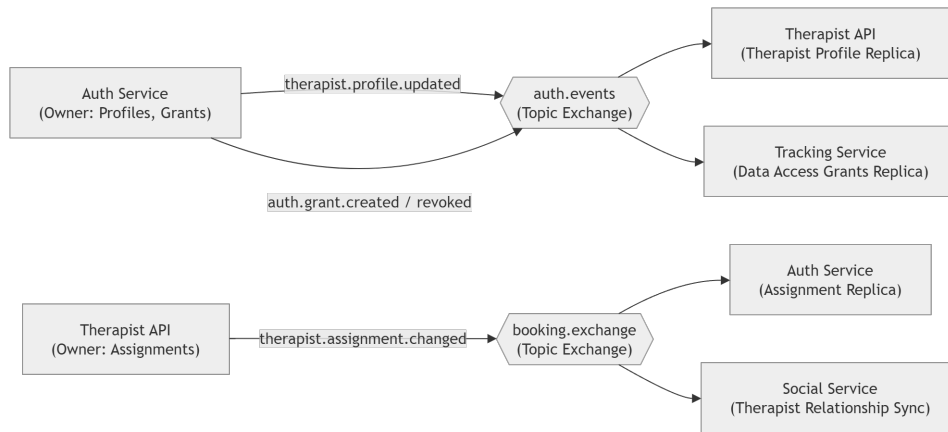


Figure 3.2: The four event-driven replication streams maintained via RabbitMQ.

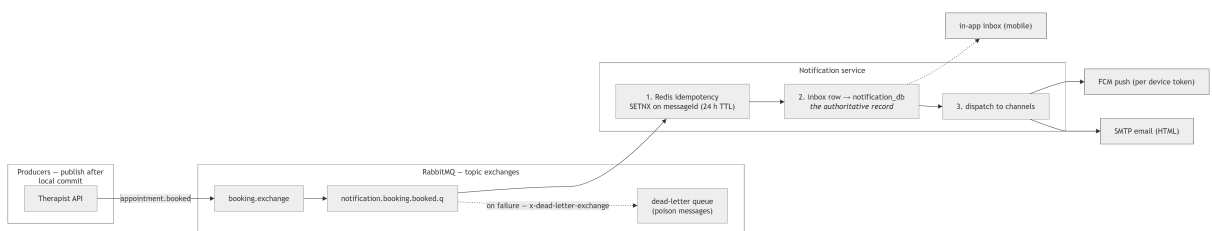


Figure 3.3: An event-flow diagram of domain event publication and consumption.

3.6 Authentication, Authorization, and Data Sharing

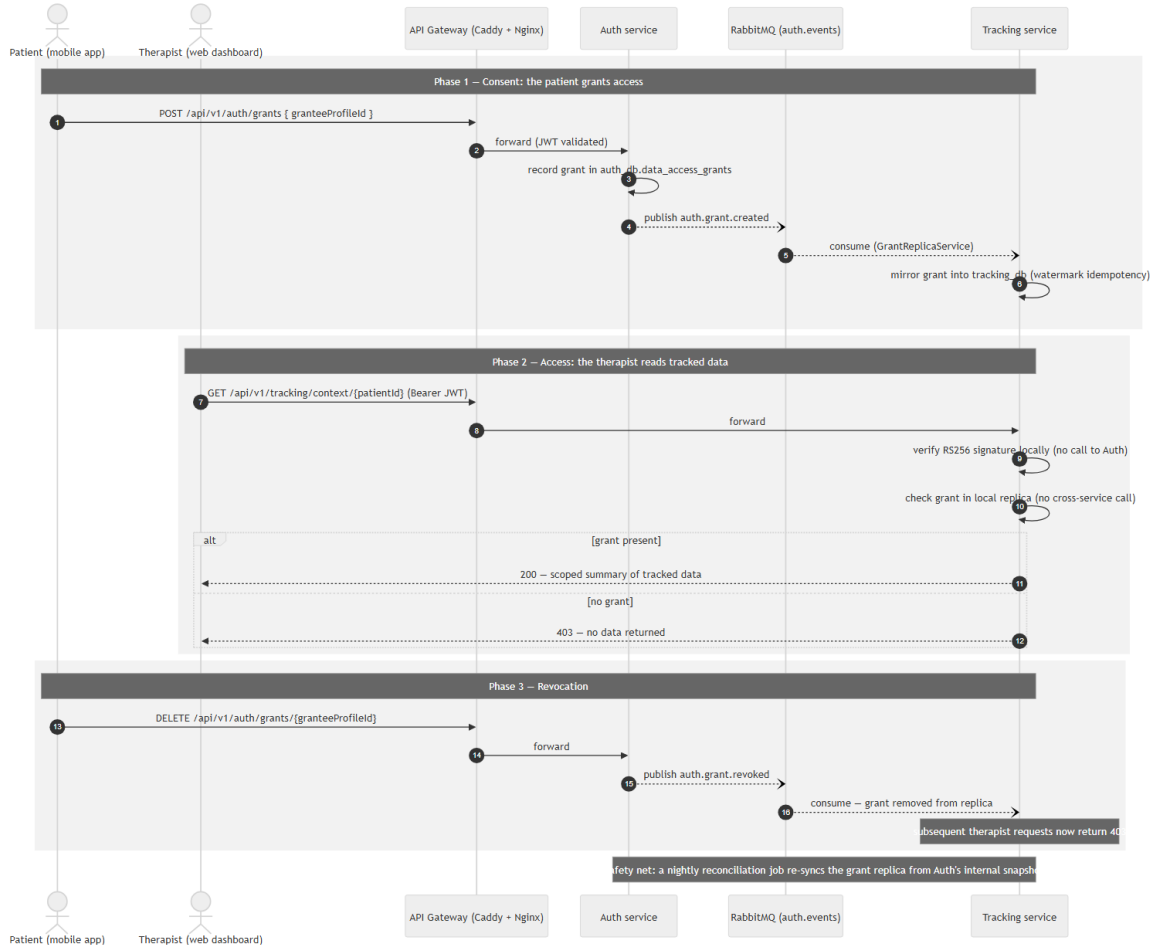


Figure 3.4: A sequence diagram of a permission-checked data access.

Authentication is stateless: the Auth service issues JSON Web Tokens signed with an RSA private key, following RFC 7519 [18], and publishes the corresponding public key through a JWKS endpoint. Every other service validates incoming tokens locally against this published key, so no service needs to call back to Auth (or share a session store) simply to authenticate a request. Roles embedded in the token (teen/patient, therapist, admin) drive coarse-grained authorization at the gateway and service level.

Fine-grained authorization is handled by a permission-based data sharing model, which is central to uMatter’s design goal of not exposing sensitive tracked data by default. Figure 3.4 traces a grant through its whole lifecycle: consent, a permission-checked read, and revocation. A user’s mood, sleep, food, journal, step, and breathing records are private to that user unless the user explicitly grants another party access. A grant names a *set of tracking categories* it covers, so a user may share, for example, their sleep and nutrition but withhold their journal, carries a thirty-day expiry, and is revocable at any time.

Enforcement lives at the Tracking service boundary: every read of a category is gated on an active, unexpired grant whose scope set includes that category, checked against a locally replicated copy of the grants (Section 3.5) rather than left to the client application, so that a compromised or misbehaving client cannot bypass it. The same mechanism serves every audience uniformly: a therapist, a parent, a friend, and the AI companion are all grantees enforced by identical code.

Service-to-service calls follow a two-tier trust model. User-facing APIs (`/api/**`) always require a valid JWT, validated locally by the receiving service. Internal APIs (`/internal/**`), such as the Tracking endpoint the AI service calls to assemble conversation context, do not carry the user’s token; the caller passes the subject’s profile identifier explicitly. These endpoints are protected by network topology instead: they are reachable only on the private container network, and the gateway refuses any external request whose path begins with `/internal/`. Asynchronous messaging is likewise authenticated at the broker with per-deployment RabbitMQ credentials. The trade-off is that any process on the internal network is trusted; mutual TLS is noted as future hardening, but on a single host not shared with third-party workloads, perimeter trust is proportionate.

3.7 AI Companion

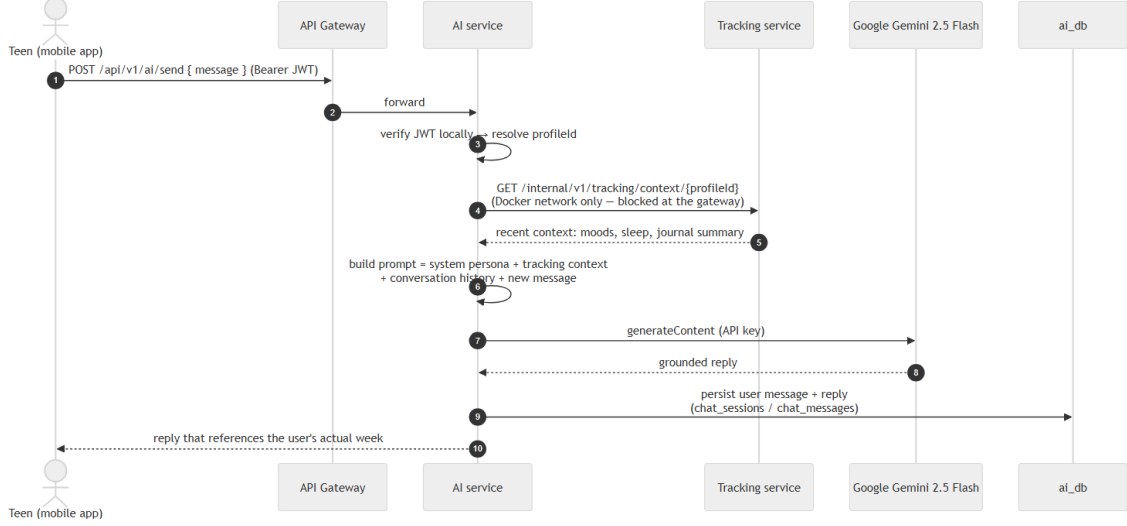


Figure 3.5: A sequence diagram of AI grounding.

The AI companion is implemented in the AI service, and Figure 3.5 traces how a reply is grounded. It retrieves the user’s recent tracking summary from the Tracking service and includes it as context when calling Google’s Gemini 2.5 Flash model [24]. Grounding the model in the user’s own data lets the companion respond to something concrete (“you logged trouble sleeping three nights this week”) rather than only offering generic supportive text. Crucially, this retrieval is gated by the same data-access grant model as every other audience (Section 3.6). Consent is off by default: a user who has not opted in still receives a companion, just one without personal grounding.

Two safeguards run on every message before and around this grounding, and both are deliberately narrow. First, user messages pass a lightweight keyword guardrail: a message matching a small list of Vietnamese self-harm or violence phrases short-circuits the model entirely and returns a fixed response directing the user to emergency hotlines, rather than being sent to Gemini. Second, personally identifying details (emails, phone numbers,

and self-introduced names) are scrubbed from the message, history, and context before they leave the platform, and chat contents are encrypted at rest. What the AI service deliberately does not do is any form of clinical risk assessment or automated escalation to a third party (no scoring, no alerting a therapist or family member), since such feature would require the clinical collaboration as discussed in Chapter 5. The keyword guardrail is a within-conversation safety redirect, not a diagnostic claim.

3.8 Professional Workflows

The Therapist service is the only feature area with a second client. Therapists work from a React web dashboard, served at the same origin as the API, covering appointments, availability, a patient roster and profile, clinical notes, the video session, and their side of the chat in Section 3.9.

A short intake form (reasons for seeking support, prior counseling, communication style, an optional LGBTQ+ priority) drives matching, which ranks therapists by the overlap between the user’s reasons and the challenges each therapist treats, relaxing the style filter and then the exclusion of past therapists rather than returning nothing. Confirming a match writes an **ACTIVE** assignment and emits the event that provisions the patient–therapist chat channel (Section 3.5). Availability is published as weekly templates, which a scheduled job expands idempotently into thirty days of bookable slots.

Booking then follows an explicit state machine. A request stays **REQUESTED** until the therapist accepts it and expires two hours before the start time: a sweep every minute auto-rejects stragglers, and the accept and reject handlers re-check that deadline so a click landing inside that minute is still refused. Confirmed appointments freeze an hour before the start, and cancellation releases the slot for re-booking. Opening the room moves the appointment to **IN_PROGRESS**. Two requests for one slot must not both succeed; Chapter 4 describes the resolution.

Afterwards the therapist files a SOAP-structured note with risk flags, editable as a draft and immutable once finalized. Because the review and the note arrive in either order, completion is two-sided: whichever lands first marks the appointment `PATIENT_COMPLETE` or `PROFESSIONAL_COMPLETE` and the second `OVERALL_COMPLETE`, with 24 hours to file a note once a review has arrived. Ownership checks tie every endpoint to the assigned therapist, but an assignment buys only this professional record; tracked data still needs a grant (Section 3.6), and the dashboard keeps the patient’s diary, food, sleep, and mood tabs locked until one exists.

Consultations run on Jitsi Meet [25]. The dashboard opens the room beside the patient’s stated reason, their permitted diary entries, and a scratchpad that feeds the note; the mobile app embeds the same room in a `WebView` whose injected scripts pin the interface to Vietnamese, hide the Google moderator sign-in, replace the lobby with a Vietnamese waiting screen, and confine navigation to the room so hanging up returns the user to the app. Both clients name the room after the appointment’s UUID, unguessable and impossible to enumerate, but a capability URL rather than admission control: anyone holding the link can join. A self-hosted Jitsi with JWT-gated entry is left to Chapter 5.

3.9 Peer and Family Connection

The Social service manages friend relationships and real-time chat over STOMP/WebSocket. A parent participates through the same mechanism as anyone else: a parent account (distinguished only by an account type recorded at registration) connects to their child as a friend, and the connection itself exposes no tracked data whatsoever.

Visibility is governed exclusively by the grant model of Section 3.6. From a friend’s profile screen, a teen may extend that same scoped grant to the friend, whether a parent or a peer; the friend then sees only the granted categories of mood and wellbeing data, and nothing after revoca-

tion or expiry. There is deliberately no privileged parental path and no automatic alerting: our baseline study found that an automatic crisis alert to family was rejected outright as pressure to perform a better mood than the one being felt, and the professionals we consulted advised that family connection be explicit and user-controlled. A parent therefore knows exactly what their child has chosen to show them, which is the intended outcome.

3.10 Self-Care Features

Focus Mode. Check-in prompts are scheduled entirely on the device: enabling Focus Mode pre-schedules a batch of local notifications at 90-minute intervals, and the queue is topped up whenever the application is opened. The Quiet Hours window (22:00–07:00 by default, user-configurable) is applied at scheduling time and stored only on the device.

Streaks and the mood calendar. Streaks are computed server-side by the Tracking service as entries arrive, per tracking category, and the mood calendar is a read view over the same store. Cross-category trend summaries shown on the home screen are assembled by the Dashboard service, which reads Tracking through an internal aggregation API rather than owning any data of its own.

Treasure Box. Stored positive memories are Tracking-service records whose photo attachments live in MinIO, and they are surfaced along the emotion-routing path of Section 3.1 (FR5): a user logging sadness is offered their saved memories, a user logging anger a breathing exercise.

Emergency path is a dedicated screen reachable from anywhere in the application in one tap. It is intentionally static client-side content: support

and hotline resources that render without any network call, so the screen stays available even if the backend is unreachable.

3.11 Deployment Topology

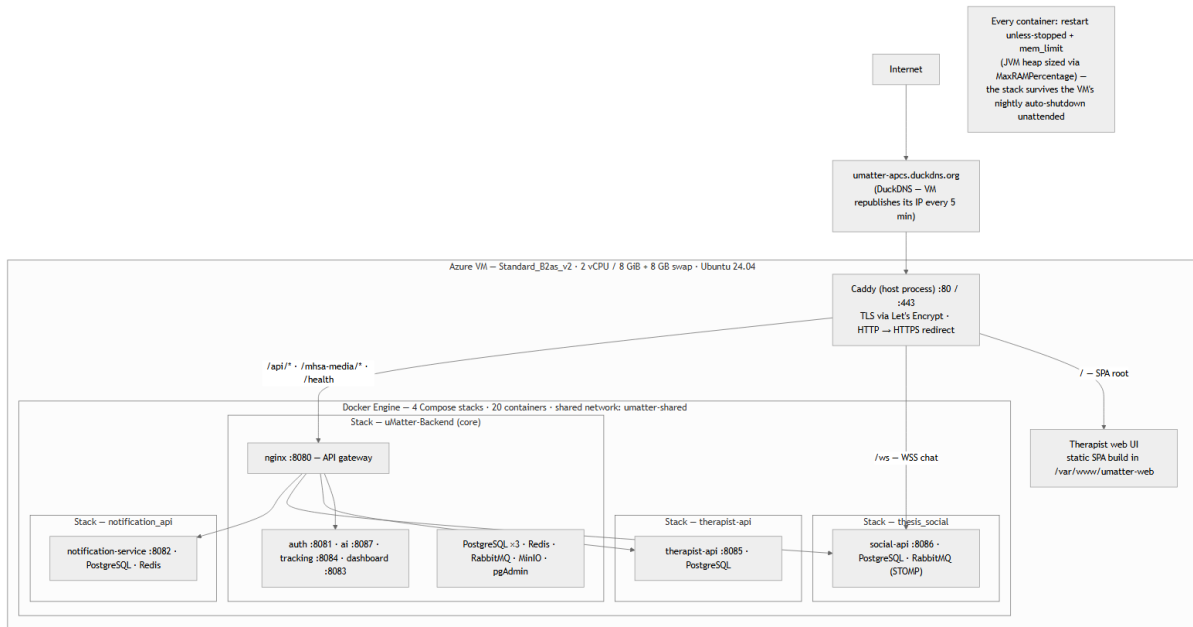


Figure 3.6: A deployment diagram of the uMatter production topology.

All services are packaged as Docker containers [17] and orchestrated with Docker Compose, grouped into stacks by repository, as shown in Figure 3.6. The full system (Spring Boot services, PostgreSQL instances, RabbitMQ, Redis, MinIO, and supporting tools such as pgAdmin) runs on a single cloud virtual machine, fronted by a reverse proxy that terminates HTTPS. The concrete deployment used for evaluation, including the migration between cloud providers, is described in Chapter 4.

Chapter 4

Implementation and Evaluation

This chapter describes how the design in Chapter 3 was implemented, the cloud deployment used to run and evaluate the system, the distribution of the mobile client, the security-hardening process applied to the platform, and an evaluation of the resulting system.

4.1 Implementation Overview

The backend services are implemented in Java 17. The Auth, Tracking, AI, and Dashboard services are built on Spring Boot 4.0 and organized as a Maven multi-module project so that shared code (for example, JWT validation logic) is written once and reused. The Therapist service is a separate Gradle project on Spring Boot 4.0, while the Social and Notification services are Gradle projects on the slightly earlier Spring Boot 3.3, reflecting when they were added to the system. Persistence throughout uses Spring Data JPA over PostgreSQL. The Social service's real-time chat is implemented with STOMP over WebSocket, and the Notification service integrates Firebase Cloud Messaging for push delivery in addition to email and an in-app inbox.

On the client side, the mobile application for teenagers is built with React Native 0.83 and React 19 in TypeScript, and the therapist-facing web dashboard is built with React 18, TypeScript, Vite, and Tailwind

CSS.

4.2 Deployment

uMatter was originally deployed on Oracle Cloud Infrastructure (OCI). During the course of this thesis, the deployment was migrated in full to Microsoft Azure, under an Azure for Students subscription. The migration involved:

- Resolving Azure region-policy restrictions that initially blocked the target subscription from provisioning a virtual machine in the desired region.
- Configuring Network Security Group (NSG) rules to open the ports required by the API gateway and by directly-exposed services (for example, RabbitMQ’s management port during setup, and ports 80 and 443 for the reverse proxy).
- Working around Azure Cloud Shell setup issues encountered while preparing the target VM.
- Reproducing the auto-shutdown behavior available on the previous provider using Azure’s built-in scheduled auto-shutdown, to control cost under the student subscription, with the containers configured to restart automatically on boot so they come back up automatically when the VM is next started.

The resulting deployment runs on a single, right-sized Azure VM hosting 20 containers: multiple Spring Boot JVM services, several PostgreSQL instances (one per owning service), RabbitMQ, Redis, MinIO, and pgAdmin. HTTPS routing is provided by Caddy as a reverse proxy, with DuckDNS supplying a stable hostname for the VM’s dynamic address, and the VM’s public IP configured as static so the hostname does not

need to be re-pointed after the nightly auto-shutdown cycle. This mirrors the general topology described in Chapter 3, with Caddy taking the reverse-proxy/HTTPS-termination role and Nginx continuing to serve as the internal API gateway between the proxy and the backend services.

4.2.1 Mobile Application Distribution

The mobile client for teenagers/patients was packaged and published to the Google Play Store, rather than distributed as a manually installed APK. Publishing through Google Play means end users install and update the app the same way as any other Android application, without needing to trust or manually side-load a build shared outside the store.

This distribution choice also shaped a concrete backend requirement: the published app communicates with the API gateway exclusively over HTTPS on port 443, because modern Android builds block plaintext HTTP traffic by default. Terminating TLS correctly at the reverse proxy, using a Let's Encrypt certificate obtained over port 80, was therefore not an optional hardening step but a hard prerequisite for the Play Store build to reach the backend at all. This is why ports 80 and 443 were treated as non-negotiable when the NSG rules were configured.

4.3 Security Hardening

Because uMatter handles sensitive personal data and coordinates real appointments, the implemented services were subjected to structured security audits organized around the OWASP Top 10 risk categories [20], covering three areas in particular:

- **Business logic correctness:** verifying that workflows such as matching, booking, and permission granting behave correctly under normal and adversarial input, not only in the happy path.

- **Access control:** specifically JPQL injection patterns in hand-written queries, insecure direct object references (for example, a user supplying another user’s record identifier to an endpoint that does not re-check ownership), and mass assignment (a client supplying extra fields in a request body that get bound to fields the client should not be able to set, such as a role or a permission flag).
- **Data integrity:** ensuring that concurrent requests cannot leave the system in an inconsistent state.

The most significant data-integrity issue identified was a race condition in appointment booking: two concurrent requests to book the same therapist for the same time slot could both pass an application-level availability check before either write was committed, resulting in a double booking. This was closed not by adding an application-level lock, which would not be safe against multiple service instances, but by making the slot claim a single atomic database operation: booking a slot is a conditional update that sets the slot’s booked flag to true only where it is still false, so exactly one of two concurrent requests updates a row and the other is rejected. A unique constraint on the appointment’s slot reference backs this up as a second line of defense. Because the guarantee lives in the database itself, it holds regardless of how many instances of the Therapist service are running concurrently.

The most significant access-control finding was in the AI service’s own path to a user’s tracking data, rather than in a caller manipulating another user’s identifier. The Tracking service’s internal context endpoint, used by the AI service to retrieve grounding data, was reachable by any authenticated internal caller. It performed no check that the caller had actually been granted access to that data, relying instead on network-level trust between services. This is precisely the class of implicit trust that the permission-based data-sharing model in Section 3.6 is designed to remove. The fix followed the same pattern used elsewhere in the codebase:

the AI companion was added as a reserved grantee in the existing access-grant table, and the context endpoint now checks for an active grant before returning anything, exactly as it already did for a therapist. No new authorization mechanism was introduced; an existing one was extended to a caller that had previously been exempted from it.

4.4 Evaluation

The system was evaluated along several dimensions: functional completeness against the design goals in Chapter 1, security posture after hardening, feature positioning relative to the competitive landscape reviewed in Chapter 2, and qualitative feedback from trial users and mental-health professionals, including the features they valued most and the wider findings that emerged from the design, trial, and consultation process.

4.4.1 Functional Completeness

All seven backend services described in Chapter 3 are implemented and deployed, and both client applications (mobile for teens, web for therapists) are functional against them. Table C.1 in Appendix C traces each requirement from Section 3.1 to its implementation status.

Every requirement is met except FR8: all three notification channels (push, email, and the in-app inbox) work and fire end-to-end for appointment booking, but the other domain events intended to raise notifications are not currently wired to producers, so coverage across events is partial.

4.4.2 Security Posture

The structured audits identified and closed concrete issues in each of the three areas examined (business logic, access control, and data integrity), including the appointment-booking race condition described above. This

does not constitute a formal, independent penetration test, and should be read as an internal hardening pass rather than a certification of the system’s security; further external review would be a reasonable next step before any production use involving real users.

4.4.3 Comparison with Existing Platforms

Revisiting the comparison in Table 2.1, the implemented system delivers on the feature combination that motivated this thesis: self-care tracking, an AI companion grounded in the user’s own data, and peer social features, alongside licensed-therapist matching, booking, video consultation, and clinical notes, connected by a permission-based sharing model that none of the compared single-purpose platforms offer in combination. The clearest gap relative to mature commercial platforms such as BetterHelp and Talkspace [7], [8] is operational maturity: those platforms have been hardened and scaled in production for years, whereas uMatter, as a thesis project, has been validated on a single-VM deployment and an internal security-audit process rather than at production scale.

4.4.4 Most-Valued Features

The Treasure Box (Hộp Trân Quý)

The Treasure Box is a personal store of positive “core memories” (photos, kind messages, small wins, life goals, and motivating reflections) that a user saves ahead of time and reopens whenever they need comforting. On a low day, when negative bias makes good memories hard to recall, this pre-saved anchor gives users self-compassion they can reach for exactly when they need it. The feature grew directly out of expert collaboration: Miss Pham Thi Thanh Qui first suggested reconnecting users with their reasons for living when feeling low, and mister Vuong Nguyen Toan Thien, CEO of LUMOS, concretized that idea into the Treasure Box, grounding it in

research on positive memory and mood. Within the app it is one destination in uMatter’s “psychology first-aid” response to a low mood, alongside guided breathing and the SOS crisis contacts.

Daily Reflective Tracking

Daily reflective tracking is the everyday self-check-in habit at the heart of the app: mood check-ins, the emotional journal, logs for sleep, nutrition, and water, automatic step counting, and guided breathing, each surfaced as a friendly “today” card with weekly trends. Doctor Pham Tran Thanh Nghiep assessed these features as “already very good” and helpful to users’ mental wellbeing, and Miss Qui, holding that physical and mental health are inseparable, endorsed the physical-health tracking (steps, sleep, nutrition, breathing) as a legitimate mental-health signal. Gentle Focus-Mode reminders roughly every ninety minutes, daily achievement “trophies” for a completed chain of self-care tasks, and streaks keep it a low-friction habit rather than a chore. The same daily data also grounds the AI companion, the therapist’s view (under permission), and the trusted-friend view, so the feature most valued on its own is also what makes the rest of the platform work.

4.4.5 Key Findings

Findings from the Design and Development Process

- **Unifying self-care and therapy is feasible behind one permission model.** A single microservices platform can serve both halves of the care journey without merging the data, as long as sharing is explicit, granular, and revocable.
- **Network-level trust between services is not enough.** The most instructive security finding was that an internal caller (the AI service) could reach a user’s tracking data with no explicit grant, meaning

that implicit inter-service trust is exactly the class of assumption the permission model must remove, even for internal callers.

- **Grounding the AI in the user’s own data**, with a safe fallback, is what makes it useful without over-reaching. Toggling the grant changes the companion’s replies materially: without it the AI states plainly that it has no tracking data, while with it the same question draws a reply reflecting the user’s actual sleep, mood, and nutrition patterns. The grounding is nevertheless *qualitative*. The companion characterizes the week (“your sleep has not been deep or long enough”) but never quotes a figure, even when explicitly asked for one. Two deliberate choices produce this: the system prompt forbids reciting the context mechanically, to keep the tone conversational rather than clinical, and the context passed to the model is itself an aggregate summary (an average, a count of poor nights, a set of mood tags) carrying no per-day values or dates. The companion is therefore demonstrably personalized but not *verifiable*: a user cannot check a reply against the numbers they logged.

Findings from User Feedback

- **Daily journaling beats periodic surveys.** Trial users found the weekly/monthly surveys redundant next to the daily journal and 3-hourly mood check-ins; the surveys were dropped. Finding: high-frequency, low-friction daily logging yields richer and more meaningful data than periodic questionnaires.
- **The most-loved features were the Treasure Box and daily tracking** (Section 4.4.4),

Findings from Expert Consultations

- **Miss Qui** endorsed the option-based emotional journal, the gentle 3-hourly mood check-ins (for complete, meaningful data and for helping users pay attention to themselves), the 4-7-8 breathing exercise as a response to anger, the reward/trophy loop for completing a chain of self-care tasks, the revocable trusted-person sharing, and physical-health tracking such as steps.
- **Mister Thien** praised the Treasure Box concept, the emotion timeline for complete mental-health data, the “psychology first-aid” bundle for negative moods (breathing, SOS contact, Treasure Box), and the SOS hotline page.
- **Doctor Nghiep** gave the strongest endorsement of the daily reflective tracking features (“already very good”).

Chapter 5

Conclusion and Future Work

5.1 Conclusion

This thesis set out to design, implement, and evaluate uMatter, a mental-health support platform for teenagers that unifies self-care tracking with professional therapy access, addressing a gap identified between existing self-care applications (Daylio, Bearable) and existing therapy platforms (BetterHelp, Talkspace, Wysa), none of which combine both halves of the care journey behind a single, permission-based data-sharing model.

The resulting system is a seven-service microservices architecture behind an API gateway, with a database-per-service data layer, stateless JWT authentication. A structured security-hardening process identified and resolved concrete issues across business logic, access control, and data integrity, database-level race condition. The full stack was deployed to cloud infrastructure, and evaluated along several dimensions.

This work shows that a single, security-conscious microservices platform can plausibly serve both sides of adolescent mental health support, rather than requiring a teenager to use many disconnected products.

5.2 Limitations

First, this is a software engineering thesis, not a clinical study: no claims are made about the platform’s clinical efficacy, and features that would require clinical judgment were explicitly excluded. Second, the security hardening performed was an internal, structured audit rather than an independent third-party penetration test. Third, the deployment evaluated in Chapter 4 runs on a single virtual machine and has not been load-tested at a scale representative of real-world production traffic.

5.3 Future Work

1. **Clinical collaboration.** Any safety-oriented feature that acts on self-reported mental-health data, most notably automated risk detection for at-risk users, requires input from licensed mental-health professionals before it can be responsibly designed, and is intentionally left out of this thesis pending that collaboration.
2. **Safety-oriented features more broadly.** Beyond risk detection, this includes guardian-visibility controls appropriate for a teenage user base, and clinical review of the AI companion’s behavior in sensitive conversations.
3. **Operational maturity.** Independent security review beyond the internal audits performed here, load and scalability testing of the current single-VM deployment, and a multi-instance or managed-database deployment topology to remove the single VM as a point of failure.

These directions would move uMatter from a functioning thesis prototype toward a platform that could be responsibly evaluated with real teenage users under appropriate clinical oversight.

References

English

- [1] Báo Thanh Niên. “Học sinh lớp 9 tại tp.hcm nhảy lầu tự tử vì điểm kém (ninth-grader in hcmc jumps to death due to poor grades),” Accessed: Jul. 17, 2026. [Online]. Available: <https://thanhnien.vn/hoc-sinh-lop-9-tai-tphcm-nhay-lau-tu-tu-vi-diem-kem-185695396.htm>
- [2] Báo Dân Trí. “Nam sinh viên tử vong dưới chân tòa nhà trường đại học (male university student dies at the foot of university building),” Accessed: Jul. 17, 2026. [Online]. Available: <https://dantri.com.vn/thoi-su/nam-sinh-vien-tu-vong-duoi-chan-toa-nha-truong-dai-hoc-20240227201657100.htm>
- [3] T. Pham, N. Nguyen, et al., “The prevalence, correlates and functions of non-suicidal self-injury in vietnamese adolescents,” *International Journal of Environmental Research and Public Health*, vol. 18, no. 22, p. 11993, 2021. DOI: 10.3390/ijerph182211993 [Online]. Available: <https://pmc.ncbi.nlm.nih.gov/articles/PMC8636691/>
- [4] World Health Organization. “Mental health of adolescents,” Accessed: Jul. 17, 2026. [Online]. Available: <https://www.who.int/news-room/fact-sheets/detail/adolescent-mental-health>
- [5] Daylio. “Daylio - micro diary and mood tracker,” Accessed: Jul. 13, 2026. [Online]. Available: <https://daylio.net>

- [6] Bearable. “Bearable - symptom and mood tracker,” Accessed: Jul. 13, 2026. [Online]. Available: <https://bearable.app>
- [7] BetterHelp. “BetterHelp - online therapy & counseling,” Accessed: Jul. 13, 2026. [Online]. Available: <https://www.betterhelp.com>
- [8] Talkspace. “Talkspace - online therapy & psychiatry,” Accessed: Jul. 13, 2026. [Online]. Available: <https://www.talkspace.com>
- [9] Wysa. “Wysa - ai-powered mental health support,” Accessed: Jul. 13, 2026. [Online]. Available: <https://www.wysa.com>
- [10] Zalo. “Zalo - talk to your love,” Accessed: Jul. 20, 2026. [Online]. Available: <https://vng.com.vn/news/product/vng-zalo.html>
- [11] Life360. “Life360 - when they’re okay, you’re okay,” Accessed: Jul. 20, 2026. [Online]. Available: <https://intl.life360.com/>
- [12] R. K. McBain et al., “Use of generative ai for mental health advice among us adolescents and young adults,” *JAMA Network Open*, vol. 8, no. 11, e2542281, 2025. DOI: 10.1001/jamanetworkopen.2025.42281
- [13] S. Newman, *Building Microservices: Designing Fine-Grained Systems*, 2nd. O’Reilly Media, 2021.
- [14] R. T. Fielding, “Architectural styles and the design of network-based software architectures,” Ph.D. dissertation, University of California, Irvine, 2000.
- [15] Broadcom Inc. “RabbitMQ - reliable messaging with erlang,” Accessed: Jul. 13, 2026. [Online]. Available: <https://www.rabbitmq.com>
- [16] VMware Tanzu. “Spring boot reference documentation,” Accessed: Jul. 13, 2026. [Online]. Available: <https://docs.spring.io/spring-boot/index.html>

- [17] Docker Inc. “Docker - accelerated, containerized application development,” Accessed: Jul. 13, 2026. [Online]. Available: <https://www.docker.com>
- [18] M. B. Jones, J. Bradley, and N. Sakimura, *JSON web token (JWT)*, IETF RFC 7519, 2015. Accessed: Jul. 13, 2026. [Online]. Available: <https://www.rfc-editor.org/rfc/rfc7519>
- [19] M. Jones, “JSON Web Key (JWK),” Internet Engineering Task Force (IETF), RFC 7517, May 2015. [Online]. Available: <https://www.rfc-editor.org/rfc/rfc7517.txt>
- [20] OWASP Foundation, *OWASP top 10:2021 - the ten most critical web application security risks*, OWASP Foundation, 2021. Accessed: Jul. 13, 2026. [Online]. Available: <https://owasp.org/Top10/>
- [21] R. S. Sandhu, E. J. Coyne, H. L. Feinstein, and C. E. Youman, “Role-based access control models,” *IEEE Computer*, vol. 29, no. 2, pp. 38–47, 1996. DOI: 10.1109/2.485845
- [22] V. C. Hu et al., “Guide to Attribute Based Access Control (ABAC) Definition and Considerations,” National Institute of Standards and Technology, NIST Special Publication 800-162, 2014. DOI: 10.6028/NIST.SP.800-162
- [23] J. H. Saltzer and M. D. Schroeder, “The protection of information in computer systems,” *Proceedings of the IEEE*, vol. 63, no. 9, pp. 1278–1308, 1975. DOI: 10.1109/PROC.1975.9939
- [24] Google DeepMind, *Gemini: A family of highly capable multimodal models*, Technical report, 2024. Accessed: Jul. 13, 2026. [Online]. Available: <https://arxiv.org/abs/2312.11805>
- [25] Jitsi. “Jitsi Meet - secure, simple, and scalable video conferencing,” Accessed: Jul. 13, 2026. [Online]. Available: <https://jitsi.org>

Appendix A

API Endpoint Summary

This appendix lists a representative subset of the REST endpoints exposed by each uMatter service. It is intended as a quick-reference companion to the full API reference maintained in the project documentation, not a complete specification.

Table A.1: Representative endpoints by service

Service	Example endpoint	Purpose
Auth	POST /api/auth/login	Authenticate a user and issue a JWT access/refresh token pair
Tracking	POST /api/tracking/mood	Record a mood/sleep/journal entry for the authenticated user
AI	POST /api/ai/chat	Send a message to the AI companion, grounded in the user’s tracking context
Dashboard	GET /api/dashboard/summary	Aggregate tracking data into a user-facing summary view
Therapist	POST /api/therapist/appointments	Book an appointment with a matched therapist
Social	GET /api/social/feed	Retrieve a user’s peer social feed
Notification	GET /api/notifications	Retrieve a user’s in-app notification inbox

Appendix B

Deployment Configuration Notes

This appendix summarizes the cloud deployment topology referenced in Chapter 4: a single Ubuntu virtual machine running the full container stack (application services, PostgreSQL instances, RabbitMQ, Redis, MinIO, and pgAdmin) behind a reverse proxy that terminates HTTPS, with DNS provided by a dynamic DNS service. Secrets and environment files are kept outside version control and are managed directly on the deployment host and in the team's private configuration store.

Appendix C

Requirement Traceability

This appendix traces each functional and non-functional requirement from Section 3.1 to its implementation status. The result is summarized in the Functional Completeness evaluation of Chapter 4.

Table C.1: Implementation status of the requirements in Section 3.1

ID	Requirement (summary)	Status
FR1	Mood check-in, journal, mood calendar, and streaks	Met
FR2	Sleep, nutrition, water, and breathing logs; automatic step count	Met
FR3	Focus Mode check-in prompts, silent during Quiet Hours	Met
FR4	AI companion grounded in tracking data only under an active grant	Met
FR5	Guided breathing and meditation, Treasure Box, one-tap emergency area	Met
FR6	Therapist discovery, booking, video consultation, and clinical notes	Met
FR7	Friend and parent connections with real-time chat	Met
FR8	Push, email, and in-app notifications from domain events	Partial
NFR1	Privacy by default, grants enforced server-side	Met
NFR2	Stateless JWT validation against a published JWKS key	Met
NFR3	Integrity under concurrent booking	Met
NFR4	Independent services on a single cloud VM	Met